Importing Required Packages

Nama : Wianda Rafa Afifah

Kelas : 4KA01

NPM : 11121292

```
!pip install mido
```

```
Collecting mido
  Downloading mido-1.3.3-py3-none-any.whl.metadata (6.4 kB)
Requirement already satisfied: packaging in /usr/local/lib/python3.10/dist-packages (from mido) (24.2)
Downloading mido-1.3.3-py3-none-any.whl (54 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 54.6/54.6 kB 2.2 MB/s eta 0:00:00
Installing collected packages: mido
Successfully installed mido-1.3.3
```

```
import mido
from mido import MidiFile, MidiTrack, Message
from keras.layers import LSTM, Dense, Activation, Dropout, Flatten
from keras.preprocessing import sequence
from keras.models import Sequential
from tensorflow.keras.optimizers import Adam
from sklearn.preprocessing import MinMaxScaler, StandardScaler
import numpy as np
```

Load MID file/files

```
#download zip dari vclass, upload hasil download ke /content/
```

```
!unzip /content/chillhopdata.zip -d chillhop
```

```
unzip:  cannot find or open /content/chillhopdata.zip, /content/chillhopdata.zip.zip or /content/chillhopdata.zip.ZIP.
```

```
import os

notes = []
for song in os.listdir("/content/chillhop"):
  mid = MidiFile('/content/chillhop/' + song)
  for msg in mid:
    if not msg.is_meta and msg.channel == 0 and msg.type =='note_on':
      data = msg.bytes()
      notes.append(data[1])
```

Scale Data

```
scaler = MinMaxScaler()
notes = list(scaler.fit_transform(np.array(notes).reshape(-1, 1)))
```

Create Train Data

```
notes = [list(note) for note in notes]

X = []
y = []
```

```
n_prev=30

for i in range(len(notes) - n_prev):
  X.append(notes[i:i+n_prev])
  y.append(notes[i+n_prev])

X_test = X[-300:]
X = X[:-300]
y = y[:-300]
```

Build LSTM

```
model = Sequential()
model.add(LSTM(256, input_shape=(n_prev, 1), return_sequences=True))
model.add(Dropout(0.6))
model.add(LSTM(128, input_shape=(n_prev, 1), return_sequences=True))
model.add(Dropout(0.6))
model.add(LSTM(64, input_shape=(n_prev, 1), return_sequences=False))
model.add(Dropout(0.6))
model.add(Dense(1))
model.add(Activation('linear'))
model.summary()

optimizer = Adam(learning_rate=0.001)
model.compile(loss='mse', optimizer=optimizer)
```

```
/usr/local/lib/python3.10/dist-packages/keras/src/layers/rnn/rnn.py:204: UserWarning: Do not pass an `input_shape`/`input_dim` argument
  super().__init__(**kwargs)
```

Model: "sequential"

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 30, 256)	264,192
dropout (Dropout)	(None, 30, 256)	0
lstm_1 (LSTM)	(None, 30, 128)	197,120
dropout_1 (Dropout)	(None, 30, 128)	0
lstm_2 (LSTM)	(None, 64)	49,408
dropout_2 (Dropout)	(None, 64)	0
dense (Dense)	(None, 1)	65
activation (Activation)	(None, 1)	0

Total params: 510,785 (1.95 MB)
Trainable params: 510,785 (1.95 MB)

Training

```
model.fit(np.array(X), np.array(y), batch_size=16, epochs=10, verbose=1)
```

```
Epoch 1/10
195/195 ━━━━━━━━━━━━━━━━━━━━ 32s 139ms/step - loss: 0.0276
Epoch 2/10
195/195 ━━━━━━━━━━━━━━━━━━━━ 40s 133ms/step - loss: 0.0196
Epoch 3/10
195/195 ━━━━━━━━━━━━━━━━━━━━ 26s 135ms/step - loss: 0.0169
Epoch 4/10
195/195 ━━━━━━━━━━━━━━━━━━━━ 42s 141ms/step - loss: 0.0164
Epoch 5/10
195/195 ━━━━━━━━━━━━━━━━━━━━ 41s 141ms/step - loss: 0.0170
Epoch 6/10
195/195 ━━━━━━━━━━━━━━━━━━━━ 40s 135ms/step - loss: 0.0159
Epoch 7/10
195/195 ━━━━━━━━━━━━━━━━━━━━ 43s 145ms/step - loss: 0.0155
Epoch 8/10
195/195 ━━━━━━━━━━━━━━━━━━━━ 27s 137ms/step - loss: 0.0157
Epoch 9/10
195/195 ━━━━━━━━━━━━━━━━━━━━ 27s 136ms/step - loss: 0.0147
Epoch 10/10
```

```
195/195 ━━━━━━━━━━━━━━━━━━━━ 41s 138ms/step - loss: 0.0136
<keras.src.callbacks.history.History at 0x7c1b5b9b4700>
```

Generating & Saving LSTM Music

```
prediction = model.predict(np.array(X_test))
prediction = np.squeeze(prediction)
prediction = np.squeeze(scaler.inverse_transform(prediction.reshape(-1,1)))
prediction = [int(i) for i in prediction]

mid = MidiFile()
track = MidiTrack()
t = 0
for note in prediction:
    # 147 means note_on
    # 67 is velosity
    note = np.asarray([147, note, 67])
    bytes = note.astype(int)
    msg = Message.from_bytes(bytes[0:3])
    t += 1
    msg.time = t
    track.append(msg)
mid.tracks.append(track)
mid.save('LSTM_music.mid')
```

```
10/10 ━━━━━━━━━━━━━━━━━━━━ 1s 98ms/step
```

Start coding or generate with AI.